Insecure by Design: A Hands-On Learning Platform for Web Security Training

Progress Report 4

CSIS 4495-003 | Applied Research Project

Winter 2026

Instructor: Arasanipalai Kandhadai, Padmapriya

Thilan Udahage Don | 300398798

Work Logs (Mar 10 – Mar 16)

Date	Number of Hours	Description of Work
Mar 10	1.5	Began implementing the user profile progress tracker. Created a profile page Thymeleaf template and updated the controller to fetch completed challenge data from the ChallengeProgress entity and pass it to the view. Displayed a summary of completed levels per challenge module.
Mar 11	2	Continued work on the user profile page. Added visual indicators (Bootstrap badges) for completed and incomplete challenge levels. Refined the layout and ensured the page integrates correctly with the shared navbar fragment.
Mar 11	1.5	Started implementing the "Why It Worked" Explainer feature for the SQLi module. Added a hidden explanation section to sqli.html that becomes visible upon successful challenge completion. Wrote explanatory content for Level 1 (OR-based injection) covering why the payload bypasses the login query.
Mar 12	2	Completed the "Why It Worked" Explainer content for SQLi Levels 2 and 3. Added JavaScript logic to reveal the explainer section dynamically after a correct payload is submitted, using the existing AJAX success response to trigger visibility.
Mar 13	1.5	Applied homepage polish. Fixed inconsistent spacing between the challenge cards, updated button labels for clarity, and added a brief descriptive subtitle under each card. Verified rendering across different browser viewport sizes.
Mar 13	2	Began file upload vulnerability implementation. Researched insecure file upload patterns and planned the challenge structure. Set up FileUploadController with an endpoint that accepts file uploads without validating file type or content. Created the initial fileupload.html Thymeleaf template with an upload form.
Mar 14	1.5	Friday team meeting with Allwyn. Reviewed progress on the file upload vulnerability, user profile tracker, and the "Why It Worked" Explainer. Discussed Allwyn's broken authentication module progress and aligned on remaining tasks for the week.
Mar 15	1.5	Saturday team meeting with Allwyn. Reviewed each other's implementations, identified minor integration issues between the profile page and Allwyn's broken auth module, and planned final tasks before PR4 submission.

Mar 15	2	Continued file upload vulnerability development. Implemented the insecure upload logic allowing upload of any file type including server-executable files. Added feedback messages displaying the uploaded filename. Used Claude to assist with the controller and template structure.
Mar 16	1.5	Tested the file upload challenge end-to-end. Verified that dangerous file types are accepted and that the "Why It Worked" Explainer section displays correctly after a successful upload. Finalized and checked in all implementation files and Progress Report 4.

Summary Description

During this reporting period, I worked across four areas: the user profile progress tracker, the “Why It Worked” Explainer, homepage polish, and the file upload vulnerability module. For the profile tracker, I created a new Thymeleaf profile page and updated the controller to fetch per-challenge completion data from the ChallengeProgress entity, displaying results using Bootstrap badges to clearly indicate completed and incomplete levels.

For the “Why It Worked” Explainer, I added hidden explanation sections to the SQLi challenge template that reveal dynamically after a correct payload is submitted. Each level now includes a concise explanation of why the payload succeeded, helping reinforce the learning outcome. I also applied several homepage polish improvements, including fixing spacing inconsistencies between challenge cards, refining button labels, and adding descriptive subtitles under each card.

The main implementation effort this period was the file upload vulnerability module. I set up FileUploadController with an endpoint that intentionally accepts file uploads without validating file type or content, allowing dangerous files such as server-executable scripts to be uploaded. I created the fileupload.html Thymeleaf template with an upload form and a feedback section displaying the uploaded filename. I used Claude to assist with the controller and template structure, then reviewed, adapted, and tested the generated code against our existing project architecture. End-to-end testing confirmed the challenge works correctly and the Explainer section displays as expected. Allwyn focused on the broken authentication module this period.

Repo Check-In of Implementation Completed

Since the previous progress report, I have checked the following files and updates into the GitHub repository under the Implementation folder:

- Created profile.html Thymeleaf template and updated ProfileController to display per-challenge completion status using Bootstrap badges.
- Added "Why It Worked" Explainer sections to sqli.html for all three levels, with JavaScript reveal logic triggered on AJAX success response.
- Applied homepage polish to index.html: fixed card spacing, updated button labels, and added challenge card subtitles.
- Created FileUploadController.java with an insecure file upload endpoint that accepts all file types without validation.
- Created fileupload.html Thymeleaf template with an upload form and filename feedback section.
- Progress Report 4 (this document) checked into the ReportsAndDocuments folder.

All files were committed individually under my own GitHub account, in compliance with the requirement for individual contributions.

AI Use Section

Table of AI Tools and Specific Use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition - What value did you add over and above what AI did for you?
Claude (Anthropic)	Claude Sonnet 4.6, free	Used to assist with coding the file upload vulnerability module — specifically the FileUploadController.java endpoint and the fileupload.html Thymeleaf template structure.	Reviewed all generated code for correctness against our Spring Boot project structure. Adapted the controller to match our existing routing and service conventions, verified that the intentionally insecure upload logic behaved as intended, and performed end-to-end testing of the challenge.

Appendix

Prompt History

Prompt 1 (Claude, Sonnet 4.6 free):

"I am building a Spring Boot MVC application with Thymeleaf for a cybersecurity training platform. I need to implement an intentionally insecure file upload vulnerability challenge. Can you help me create a FileUploadController with a POST endpoint that accepts file uploads without any file type or content validation, and a fileupload.html Thymeleaf template with an upload form that displays the uploaded filename as feedback?"